Università degli Studi di Udine

Dipartimento di Scienze Matematiche,
Informatiche e Fisiche

IBML - Internet of Things, Big Data, Machine Learning

Internet of Things
Progetto Pet Tracker

Autori

Andrea Gioia, 169484	169484@spes.uniud.it
Alessio Nicodemo, 166972	166972@spes.uniud.it
Mattia Nadal, 167372	167372@spes.uniud.it
Francesco Sabot, 167339	167339@spes.uniud.it

Sommario

Il progetto Pet-Tracker propone la realizzazione di un sistema IoT per il tracciamento in tempo reale di animali domestici di media taglia. La soluzione integra un dispositivo embedded basato su ESP32-S3 con connettività LTE e modulo GNSS, in grado di acquisire e trasmettere dati di posizione, stato della batteria e attività motoria verso un backend remoto.

L'architettura segue il modello IoT World Forum (IoTWF) e adotta tecniche di ottimizzazione energetica, tra cui modalità Deep Sleep e strategie di Hot Start per il GPS. I dati vengono elaborati e resi disponibili tramite API REST e visualizzati attraverso un'applicazione mobile dedicata.

L'obiettivo principale è garantire un sistema affidabile, a basso consumo energetico e scalabile per il monitoraggio continuo dell'animale.

Indice

1 Architettura del sistema **3**
1.1 Mappatura sul modello IoTWF 3

2 Hardware **3**
2.1 LiLyGo T-SIM7670G-S3 3
2.2 Antenna GPS e LTE 3
2.3 Batteria e Power Management 3
2.4 BMA400 CJMCU-400 4

3 Firmware: Logica di funzionamento **4**
3.1 Gestione della Persistenza e Hot Start 4
3.2 Protocollo di Comunicazione HTTPS 4
3.3 Risparmio Energetico e Deep Sleep 5
3.4 Monitoraggio Hardware e Batteria 5

4 Back-end e Database **6**
4.1 Architettura di Hosting e Network Exposure 6
4.2 Sistema di Backup e Data Retention 6
4.3 Monitoraggio e Business Continuity 6
4.4 Struttura dei dati 6
4.5 Interfaccia API e API Rules 7
4.5.1 API Rules e Sicurezza 7
4.6 Smistamento e Normalizzazione 7
4.6.1 Funzionamento del Trigger onRecordAfterCreateSuccess 7
4.6.2 Vantaggi Architetturali e Corrispondenza IoTWF 8

5 Front-end e Applicazione Mobile **8**

6 Test e Risultati **8**

7 Limiti del sistema **8**

8 Conclusioni **9**

Riferimenti bibliografici **10**

1 Architettura del sistema

L'architettura del sistema Pet-Tracker segue il **Modello a 7 livelli dell'IoT World Forum (IoTWF)**, garantendo una transizione fluida dal dato fisico all'informazione per l'utente.

1.1 Mappatura sul modello IoTWF

- **Livello 1 (Physical Devices):** Rappresentato dalla board LilyGo e dai sensori (GPS, Accelerometro).
- **Livello 2 (Connectivity):** Gestito tramite il modem LTE e protocolli di trasporto sicuri.
- **Livello 3 (Edge Computing):** Elaborazione locale sul microcontrollore per il filtraggio delle stringhe NMEA e il calcolo dei passi.
- **Livello 4 (Data Accumulation):** Persistenza dei dati sul server PocketBase (database SQLite).
- **Livello 5 (Data Abstraction):** Esposizione dei dati tramite API REST strutturate in formato JSON.
- **Livello 6 (Application):** L'applicazione mobile dedicata che trasforma le coordinate in posizioni su mappa.
- **Livello 7 (Collaboration and Processes):** L'interazione del proprietario con il sistema per il monitoraggio dell'animale.

2 Hardware

2.1 LiLyGo T-SIM7670G-S3

Il dispositivo utilizza una board LiLyGo T-SIM7670G-S3 [1] che si basa sul modulo **ESP32-S3** dual-core, che gestisce sia la logica applicativa che la comunicazione con il modem SIM7670G tramite interfaccia UART.

2.2 Antenna GPS e LTE

Il sistema utilizza un'antenna flessibile LTE [2] per la connettività dati e un'antenna ceramica attiva [3] per il ricevitore GNSS integrato, essenziale per la precisione della localizzazione outdoor.

2.3 Batteria e Power Management

Viene impiegata una batteria ricaricabile 3000mAh 30A Litio-Ion ad alta capacità. Il monitoraggio avviene tramite un partitore di tensione collegato al pin `ADC_BAT` (GPIO 4), permettendo di calcolare la percentuale di carica residua.

2.4 BMA400 CJMCU-400

L'accelerometro usato è un **BMA400** [4] integrato nella breadboard **CJMCU-400**, che viene interrogato per rilevare il movimento, consentendo l'implementazione di un contapassi utile al monitoraggio della salute dell'animale.

3 Firmware: Logica di funzionamento

Il firmware, sviluppato in ambiente `Arduino` per il SoC `ESP32-S3`, gestisce l'acquisizione dei dati dai sensori, il controllo del modem LTE e l'ottimizzazione energetica tramite una macchina a stati finiti.

3.1 Gestione della Persistenza e Hot Start

Per ridurre drasticamente il *Time To First Fix* (TTFF) del modulo GNSS, il firmware sfrutta la memoria **RTC (Real-Time Clock)** dell'ESP32. Questa memoria permette di mantenere i dati vitali anche durante i cicli di *Deep Sleep*. Le variabili critiche, come l'ultima posizione valida (`lastLat`, `lastLon`) e i timestamp (`lastGpsDate`, `lastGpsTime`), vengono dichiarate con l'attributo `RTC_DATA_ATTR`.

Al risveglio del dispositivo, la funzione `iniettaGps()` esegue le seguenti operazioni:

- Recupera le coordinate geografiche e temporali salvate nel ciclo precedente.
- Inietta la posizione nota nel modulo GNSS tramite il comando `AT+CGNSSPOS`.
- Sincronizza l'orario del modem con il comando `AT+CGNSSTIME`.

Questo meccanismo abilita la modalità **Hot Start**, che minimizza il tempo di ricerca dei satelliti e, di conseguenza, il consumo energetico.

```
void iniettaGps() {
  if (!hasGpsFix) return;
  // Costruzione comando per Hot Start basato su ultima posizione nota
  String cmdPos = "AT+CGNSSPOS=" + formatCoordinate(lastLat, true) + "," + latDir + "," +
                  formatCoordinate(lastLon, false) + "," + lonDir + ",0,100";
  sendAT(cmdPos.c_str());
}
```

Listing 1: Ottimizzazione del fix GPS tramite iniezione di dati persistenti

3.2 Protocollo di Comunicazione HTTPS

La trasmissione dei dati al backend PocketBase è affidata a una serie di comandi AT inviati via seriale al modem SIM7670G. Il firmware incapsula le informazioni in un oggetto JSON strutturato, inviato tramite una richiesta `HTTPS POST`. Il payload include:

- `board_id`: un identificativo univoco derivato dall'IMEI del modem;
- `geo`: un oggetto con latitudine e longitudine per la gestione cartografica;
- `battery` / `steps`: telemetria sullo stato di carica e sull'attività motoria rilevata;

- `sleep`: un flag booleano che segnala al server l'imminente entrata in stato di inattività.

```
void inviaDati(float l_lat, float l_lon, const BatInfo& bat, const String& timestamp, const StepData& step, bool isSleeping) {
  String json = "{";
  json += "\"timestamp\":\"" + timestamp + "\",";
  json += "\"geo\":{\"lon\":" + String(l_lon, 6) + ",\"lat\":" + String(l_lat, 6) + "},";
  json += "\"battery\":" + String(bat.voltage, 2) + ",";
  json += "\"sleep\":" + String(isSleeping ? "true" : "false");
  json += "}";
  // Invio tramite comandi AT HTTP
  modem.print(json);
}
```

Listing 2: Esempio di strutturazione del payload JSON

3.3 Risparmio Energetico e Deep Sleep

Il sistema è progettato per massimizzare l'autonomia della batteria da 3000mAh. La logica di risparmio energetico è coordinata con l'accelerometro **BMA400**:

1. **Rilevamento Attività**: Il firmware monitora costantemente il numero di passi. Ogni nuovo movimento resetta un timer di inattività (`lastActivityTime`).
2. **Invio Finale**: Se non viene rilevata attività per un tempo superiore a `SLEEP_TIMEOUT` (30 secondi), il sistema invia un ultimo pacchetto dati aggiornando il flag `sleep` a `true`.
3. **Ibernazione**: Il microcontrollore entra in *Deep Sleep*, spegnendo i moduli radio. Il risveglio è gestito tramite interrupt hardware sul pin `GPIO_5`, generato dal BMA400 solo al rilevamento di un nuovo movimento.

```
void enterDeepSleep() {
  Serial.println("\nEntro in Deep Sleep...");
  // Configura il pin di risveglio (GPIO 5) collegato al BMA400
  pinMode(WAKEUP_PIN, INPUT_PULLDOWN);
  esp_sleep_enable_ext0_wakeup(WAKEUP_PIN, 1);
  esp_deep_sleep_start();
}
```

Listing 3: Configurazione del Deep Sleep con risveglio tramite accelerometro

3.4 Monitoraggio Hardware e Batteria

Il firmware gestisce in modo intelligente l'alimentazione per evitare letture errate o consumi parassiti:

- **Controllo ADC**: Il circuito del partitore di tensione per la batteria viene attivato solo durante la lettura tramite il pin `BAT_ADC_EN`, garantendo la precisione del voltaggio acquisito dal pin `ADC_BAT`.

- **Stato USB**: Attraverso la lettura dei registri hardware dell'ESP32, il sistema rileva se il dispositivo è collegato a una fonte di ricarica USB, settando dinamicamente il campo `charging` nel pacchetto dati.

4 Back-end e Database

Il centro nevralgico della gestione dati è basato su **PocketBase** [5], una soluzione open-source che integra un database embedded SQLite con un'interfaccia API REST.

4.1 Architettura di Hosting e Network Exposure

L'istanza di PocketBase è stata ospitata su una workstation locale e data la natura del sistema IoT, che richiede la raggiungibilità del backend da parte del dispositivo (tramite rete LTE) e dell'applicazione mobile (tramite rete dati/Wi-Fi), si è reso necessario esporre il servizio locale su rete pubblica. Il problema del *NAT Traversal* e dell'assenza di un indirizzo IP pubblico statico sulla rete locale è stato risolto mediante l'impiego di **ngrok** [6]. Questa tecnologia di *tunneling* crea un tunnel sicuro (HTTPS) tra l'endpoint pubblico di ngrok e la porta locale su cui è in esecuzione il server. Tale URL funge da *Reverse Proxy*, reindirizzando il traffico HTTPS in ingresso verso l'istanza locale di PocketBase, garantendo così la persistenza dei dati e l'accessibilità dei servizi in tempo reale ovunque sia presente una connessione internet.

4.2 Sistema di Backup e Data Retention

Per garantire la resilienza dei dati e prevenire perdite accidentali dovute a guasti del server locale, è stato implementato un sistema di backup automatizzato. Il processo utilizza l'utility **rclone** [7], uno strumento a riga di comando per la gestione del cloud storage, configurato per eseguire la sincronizzazione della directory dei dati di PocketBase verso un bucket remoto su **Backblaze B2** [8]. L'automazione è gestita tramite un *cron job* che esegue il backup in modo incrementale ogni primo giorno del mese.

4.3 Monitoraggio e Business Continuity

Per garantire l'affidabilità dell'intero ecosistema, è stato implementato un sistema di monitoraggio esterno tramite **UptimeRobot** [9]. Dato che l'accessibilità del backend dipende dalla stabilità del tunnel ngrok, il monitoraggio interroga l'endpoint HTTPS a intervalli regolari, inviando notifiche immediate in caso di downtime.

4.4 Struttura dei dati

I dati ricevuti vengono normalizzati e memorizzati in una collezione denominata `data_sent_raw`. La struttura del record prevede i seguenti campi:

- `timestamp`: riferimento temporale dell'acquisizione.
- `lat` / `lon`: coordinate geografiche acquisite dal modulo GNSS;
- `battery`: stato di carica della batteria (espresso in voltaggio);
- `battery_percent`: stato di carica della batteria (espresso in percentuale);

- `charging`: booleano che indica se la batteria è in corso di ricarica;
- `steps`: conteggio dei passi rilevato dall'accelerometro BMA400;
- `sleep`: booleano che indica se il dispositivo è in stato di "sleep" (assenza di movimento prolungata);

4.5 Interfaccia API e API Rules

L'accesso ai dati memorizzati nel backend avviene esclusivamente tramite un'interfaccia **RESTful API** fornita nativamente da PocketBase. Questo approccio permette di separare nettamente il livello di persistenza (Database) dal livello di fruizione (Applicazione Mobile).

4.5.1 API Rules e Sicurezza

Per garantire l'integrità e la riservatezza dei dati, sono state configurate specifiche **API Rules** basate su espressioni booleane:

- **List/View:** Visualizzazione permessa solo agli utenti autenticati.
- **Create:** Abilitata per il dispositivo LilyGo tramite autenticazione dedicata.
- **Update/Delete:** Ristrette ai soli amministratori del sistema.

4.6 Logica di Smistamento e Normalizzazione (Server-side Hooks)

Per ottimizzare la gestione dei dati e garantire la separazione delle responsabilità, è stata implementata una logica di smistamento automatico lato server tramite i *JavaScript Event Hooks* di PocketBase. Questa architettura permette di mantenere la collezione `data_sent_raw` come un punto di accumulo "grezzo", delegando al server il compito di distribuire le informazioni nelle tabelle specializzate.

4.6.1 Funzionamento del Trigger onRecordAfterCreateSuccess

Il sistema sfrutta l'hook `onRecordAfterCreateSuccess`, che si attiva immediatamente dopo il salvataggio di un nuovo record inviato dalla board LilyGo. Il processo opera su tre direttrici principali:

- **Persistenza Geografica ed Energetica:** I parametri di tensione, percentuale di carica e coordinate GPS vengono estratti e duplicati nelle collezioni `battery_data` e `positions_duplicate`. L'uso di blocchi *try-catch* indipendenti garantisce che un errore in un sensore secondario non comprometta il tracciamento GPS.
- **Logica di Sessione (Activity Tracking):** Il trigger implementa una macchina a stati basata sul parametro `sleep`. Se il dispositivo è attivo, il sistema verifica l'esistenza di una sessione: in caso positivo aggiorna il conteggio passi, in caso negativo crea una nuova attività.
- **Gestione dello Stato Sleep:** Al ricezione di un pacchetto con `sleep: true`, il trigger chiude la sessione corrente impostando `is_active = false`, permettendo una segmentazione precisa dei periodi di movimento.

```
onRecordAfterCreateSuccess((e) => {
    const raw = e.record;
    const boardId = raw.getString("board_id");
    const timestamp = raw.getString("timestamp");
    const sleep = raw.getBool("sleep");
    const steps = raw.getInt("steps");

    // 1. Logica di gestione delle sessioni (Activity Tracking)
    if (!sleep) {
        if (activeActivity) {
            // Aggiorna passi e fine sessione corrente
            activeActivity.set("total_steps", currentSteps + steps);
            activeActivity.set("end_time", timestamp);
            e.app.save(activeActivity);
        } else {
            // Crea nuova sessione attiva se non esistente
            activitiesRecord.set("is_active", true);
            activitiesRecord.set("total_steps", steps);
            e.app.save(activitiesRecord);
        }
    } else if (activeActivity) {
        // Chiusura sessione per stato SLEEP del dispositivo
        activeActivity.set("is_active", false);
        e.app.save(activeActivity);
    }
    e.next();
}, "data_sent_raw");
```

Listing 4: Implementazione della logica di sessione in PocketBase

4.6.2 Vantaggi Architetturali e Corrispondenza IoTWF

Questa implementazione riflette i principi del Modello IoTWF in due punti chiave:

1. **Efficienza del Livello 3 (Edge):** La LilyGo invia un unico pacchetto "bulk", riducendo il tempo di attività del modem LTE e preservando la batteria.
2. **Ottimizzazione del Livello 5 (Data Abstraction):** L'applicazione Flutter non deve processare dati grezzi; interroga API già normalizzate dal trigger, garantendo una visualizzazione fluida della cronologia.

5 Front-end e Applicazione Mobile

L'applicazione mobile, sviluppata in Flutter [10], interagisce con i dati tramite il PocketBase SDK, garantendo una comunicazione asincrona e performante.

6 Test e Risultati

7 Limiti del sistema

Nonostante i risultati ottenuti, il sistema presenta alcune limitazioni:

- **Dipendenza dalla rete LTE:** in assenza di copertura, la trasmissione dei dati non è garantita.
- **Prestazioni indoor:** il modulo GNSS risulta poco affidabile in ambienti chiusi.
- **Backend locale:** l'utilizzo di PocketBase su macchina locale, esposto tramite ngrok, introduce dipendenze da servizi esterni e limita la scalabilità.
- **Consumo energetico del modem LTE:** la trasmissione dati rappresenta la principale fonte di consumo.

8 Conclusioni

Riferimenti bibliografici

[1] *T-SIM7670G-S3 Specifications*. URL: https://github.com/Xinyuan-LilyGO/LilyGo-Modem-Series/blob/main/docs/en/esp32s3/sim7670g-s3/REAMDE.MD.

[2] *LTE Antenna Specifications*. URL: https://github.com/Xinyuan-LilyGO/LilyGo-Modem-Series/blob/main/datasheet/LTE%20Antenna%20Specifications.pdf.

[3] *GPS Antenna Specifications*. URL: https://github.com/Xinyuan-LilyGO/LilyGo-Modem-Series/blob/main/datasheet/GPS%20Antenna%20Specifications.pdfp.

[4] *BMA400 CJMCU-400 Datasheet*. URL: https://www.bosch-sensortec.com/media/boschsensortec/downloads/product_flyer/bst-bma400-fl000.pdf.

[5] *PocketBase Documentation*. URL: https://pocketbase.io/docs/.

[6] *ngrok Documentation - HTTP Tunnels*. URL: https://ngrok.com/docs/secure-tunnels/.

[7] *rclone - rsync for cloud storage*. URL: https://rclone.org/.

[8] *Backblaze B2 Cloud Storage*. URL: https://www.backblaze.com/b2/cloud-storage.html.

[9] *UptimeRobot - Free Website Monitoring Service*. URL: https://uptimerobot.com/.

[10] *Flutter Documentation*. URL: https://docs.flutter.dev/.